表情仓

MEME VAULT

第一周项目阶段简报

从登录、单张上传、审核、列表到下载，跑通最小闭环



当前登录页：漫画柔彩拼贴视觉与真实表情素材

周期	版本	技术栈	依赖
2026.10.27 — 10.30	项目 v0.1.0 / 简报 v1.0	Node.js 22 · SQLite	0 个 npm 运行时依赖

摘要

一句话结论：第一周不是“页面搭好了”，而是把身份、内容、审核、下载四段业务连成了可部署的完整闭环。

阶段结论

- 普通用户注册后可上传单张表情，内容进入 pending 审核态；审核员通过后进入公开广场。
- 登录用户可按分类、关键词检索并下载原图；下载行为经过权限校验并累加计数。
- 已交付 SQLite 持久化、三角色权限、审核状态机、响应式界面、9 项集成测试与云端部署说明。

16	3	9	0
API 接口	**核心数据表**	**自动化测试**	**运行时依赖**
认证 / 表情 / 审核	users / sessions / emojis	后端集成断言	无第三方 npm 包

阅读导航 本简报的结构

章节	内容	对应问题
0 需求分析	背景、角色、范围、技术路线、总体架构	为什么做、做到什么程度
1 前端设计效果	视觉方向、页面结构、CSS 技术点、响应式	界面长什么样、CSS 在哪里
2 数据库设计	ER 图、三张表、索引、数据一致性	数据如何存、关系如何约束
3 公共类设计	ApiError、后端模块、前端公共函数、设计令牌	哪些能力被复用
4 关键流程与接口	认证、上传、审核状态机、16 个接口、权限矩阵	业务如何跑通
5 分工与 AI 协作	实际参与主体、阶段分工、AI 工具作用、多人团队模板	每个人做了什么
6 验收与下一步	测试记录、风险、后续路线、结论	如何证明完成
附录 A/B	运行、部署、目录索引	如何接手与上线

说明：本阶段实际采用“项目发起人 + AI 编程代理”的协作方式。第 5 章按真实参与主体记录，并附多人小组填报模板，避免虚构成员姓名。

0 需求分析

先锁定最小闭环，再围绕闭环补安全、检索和响应式体验。

0.1 项目背景与目标

团队日常在聊天工具里互发表情，文件容易过期、重复收藏严重，也缺少发布前的质量把关。项目希望提供一个团队内部的表情仓库：把“看到好图随手丢进来”和“需要时一搜就有”变成稳定流程。

首周目标不是做完整内容平台，而是验证四个关键假设：账号能区分角色；用户能上传；审核能控制发布；通过审核的内容能被检索和下载。围绕这四个假设，范围被刻意裁剪为“单张上传 + 单张下载”，批量能力留到下一阶段。

完整需求池覆盖：前端登录、批量添加、查询、下载、分类；后端审核、修改、删除；普通用户/审核员/管理员三角色；以及后续云端部署。首周把登录/注册、单张上传、列表展示、单张下载设为 Must，同时提前落地审核与分类筛选，保证演示闭环真实可用；批量添加、分类管理、全局修改/删除 UI 留在下一阶段。

> **验收标准**
>
> - 新用户能独立完成注册、登录，并在刷新后保持会话。
> - 普通用户上传后看到“待审核”；审核员登录后能看到同一条内容并通过。
> - 通过后内容出现在公共列表，其他登录用户能搜索并下载到原始文件。
> - 普通用户看不到审核台；非本人且未通过的内容不能被直接访问。

0.2 用户角色与核心诉求

角色	核心诉求	本阶段能力	边界
普通用户	快速收藏、需要时能找到	注册登录、单张上传、浏览/搜索、下载、查看自己的审核状态	不能审核、不能修改已发布内容
审核员	控制内容质量与分类	审核队列、通过/驳回、查看全部待审内容	不能删除他人内容
管理员	处理异常内容与账号	审核员全部能力；API 层可修改/删除任意表情；命令行创建账号	全局管理 UI 留到下一阶段

角色采用数据库字段 + 服务端鉴权 + 前端可见性三层一致控制。前端隐藏入口只是体验优化，真正的权限判断始终在 API 层完成。

0.3 范围与优先级

优先级	需求	状态	说明
Must	登录 / 注册 / 退出	已完成	scrypt 密码、HttpOnly 会话
Must	单张上传	已完成	四格式校验、5 MB 限制
Must	审核通过 / 驳回	已完成	仅审核员与管理员
Must	列表展示	已完成	公开列表只含 approved
Must	单张下载	已完成	鉴权、原图、计数
Should	分类 / 搜索 / 排序 / 分页	已完成	广场检索体验
Should	我的上传与状态反馈	已完成	审核备注、删除待审内容
Should	移动端适配	已完成	390×844 无横向溢出
Could	批量上传 / 批量审核	下一阶段	当前 API 明确拒绝多文件
Could	管理员全局内容管理页	部分完成	后端修改/删除已具备，UI 待做
Could	批量下载 / 分类管理	下一阶段	首周聚焦单张闭环
Won't	公开社交、付费、邮件找回	不在本期	避免稀释最小闭环

0.4 技术路线

在首周时间与后续云端部署的约束下，技术路线选择“**单进程 Node.js + SQLite + 原生前端**”。它不是规模最大的方案，但它是把依赖、构建链和运维成本压到最低的方案。

决策点	选型	理由	代价 / 后续演进
运行时	Node.js 22	内置 HTTP、FormData、SQLite、加密能力，一套语言贯通前后端	CPU 密集任务需拆分
数据库	node:sqlite	零安装、单文件、事务与索引足够首周	单写者；规模化后换 PostgreSQL
前端	原生 HTML/CSS/JS	无构建链、无 UI 框架，视觉可精细控制	组件复用靠约定与函数
认证	scrypt + 服务端会话	密码不落明文；会话可即时失效	多实例需共享会话存储
文件	本地 uploads/ + 元数据入库	读取简单、备份直观	多实例需对象存储
部署	单机 + Nginx/Caddy	首周即可上线，迁移路径清晰	容量与高可用下一阶段

关键取舍：不引入 ORM、不引入前端框架、不做多文件上传，把有限时间放在权限、审核状态、文件安全和真实闭环验收上。数据库、文件目录、会话都预留了后续替换边界。

0.5 总体架构

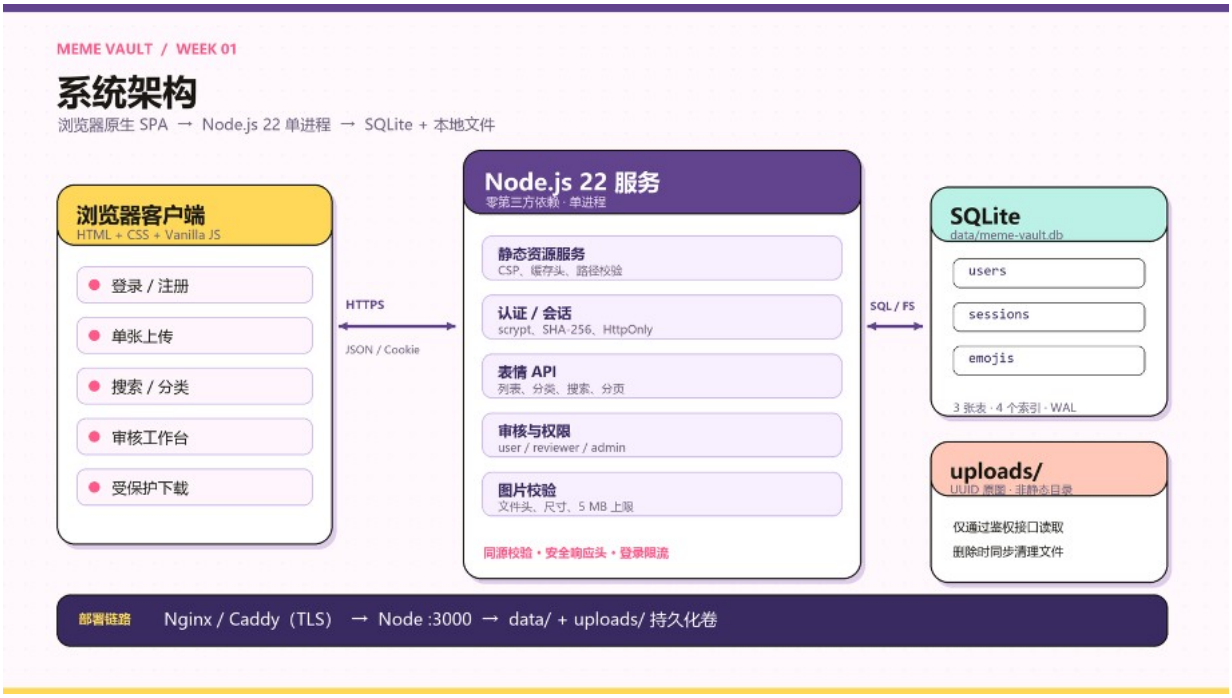


图 0-1 系统总体架构：原生客户端、Node.js 服务、SQLite 与本地文件

浏览器负责渲染与交互，所有业务请求走同源 API。Node 服务同时承担静态资源、认证、表情业务、审核权限与图片校验；SQLite 保存账号、会话和表情元数据，原图保存在非静态的 uploads/ 目录，只能经过鉴权接口读取。

0.6 非功能需求

维度	具体要求	落地方式
安全	密码不可逆、会话不可伪造、越权不可访问	scrypt；token 哈希入库；同源校验；服务端角色校验；CSP
上传安全	只接受真实图片，限制大小与尺寸	文件头识别；扩展名不信任客户端；5 MB / 8192 px 上限
性能	首屏可用、列表不一次性加载全部	索引；24 条/页；按需加载；静态资源 no-cache
兼容性	桌面与手机均可完成主流程	CSS Grid/Flex；5 个响应断点；移动底栏
可维护性	模块边界清晰、公共能力可复用	config/database/security/images 拆分；统一 ApiError
可部署性	云端单机可启动、数据可持久化	环境变量；data/ 与 uploads/ 持久卷；HTTPS Cookie

1 前端设计效果

有完整的自定义 CSS：不是套模板，而是从设计令牌、组件到动效的一整套实现。

1.1 视觉方向

当前界面采用“**漫画分镜 + 柔彩拼贴 + Y2K 贴纸**”的视觉语言：紫色渐变登录页、放射线背景、漂浮表情卡片、粗描边与硬阴影；工作台则用深紫侧栏搭配浅紫纸面，让内容卡片像贴在分镜板上的贴纸。

设计经历了迭代：先完成奶油色编辑风的基础版本，再通过追加主题层覆盖为紫粉漫画风。CSS 中保留了基础令牌，并在文件后段集中覆盖 `--bg / --accent / --mint / --yellow` 等变量，这种“主题层”写法便于后续换肤而不破坏组件结构。

- **登录页**：左右分栏，左侧品牌叙事 + 三张真实表情贴纸 + 旋转放射线；右侧表单卡片带“いらっしゃいませ”贴纸，降低工具感。
- **工作台**：252 px 深紫侧栏、吸顶工具栏、粉紫渐变 Hero、分类胶囊、5 列表情卡片；卡片使用 2 px 描边与偏移硬阴影。
- **状态色**：已发布薄荷绿、待审核鹅黄、未通过蜜桃粉；状态不只靠颜色，同时带图标和文字。

1.2 页面与组件结构

页面 / 组件	主要元素	交互反馈
登录 / 注册	分栏插画、登录/注册 Tab、字段图标、错误提示	Tab 切换、按钮 loading、错误内联提示
表情广场	Hero、排序、分类胶囊、卡片网格、加载更多、空状态	搜索防抖、分类筛选、骨架屏、hover 抬升
上传弹窗	拖拽区、文件预览、标题/分类、审核提示	拖拽高亮、预览/移除、上传按钮禁用
我的上传	四色统计卡、状态徽章、审核备注、删除按钮	审核状态一目了然、仅待审/驳回可删
审核台	角色标识、待审数量、审核卡片、通过/驳回	按钮处理中状态、驳回原因弹窗
移动端	两列卡片、底部导航、悬浮上传键	侧栏隐藏、搜索独占一行、无横向滚动

1.3 CSS 实现细节

CSS 能力	具体做法	代表位置 / 效果
设计令牌	43 个自定义属性：颜色、圆角、字体、阴影	:root 两套主题、组件统一取值
布局	Grid + Flex；侧栏、卡片网格、统计卡、双栏表单	.app-shell、.emoji-grid、.stats-grid
背景纹理	radial-gradient 圆点、conic-gradient 放射线、mask-image 渐隐	登录页与 Hero 的分镜背景
卡片语言	2 px 墨色描边 + 偏移 box-shadow + 轻微旋转	.meme-sticker、.art-card、.emoji-card
玻璃与层次	backdrop-filter、半透明遮罩、堆叠阴影	顶栏、贴纸说明、弹窗遮罩
响应式	12 组媒体查询、5 个主断点（1180/1000/900/680/340）	桌面 5 列 → 移动 2 列；侧栏 → 底栏
动效	20 个 @keyframes：入场、漂浮、闪烁、骨架屏、Toast	页面进入、贴纸浮动、卡片加载
可访问性	focus-visible、aria-label、语义按钮、prefers-reduced-motion	键盘可见焦点、降低动效
图标	内联 SVG symbol sprite，自绘 19 个图标	无外部图标库与字体请求
表单控件	自定义 input/textarea/search、错误态、拖拽态	统一 13–18 px 圆角与焦点光环

量化结果：独立 `public/styles.css` 约 **62.9 KB / 3,587 行**，包含 **393 个类选择器、43 个设计令牌、20 个关键帧、12 组媒体查询。**没有 Tailwind、Bootstrap、组件库或外部字体。

1.4 页面效果



图 1-1 表情广场：粉紫 Hero、分类与五列卡片



图 1-2 上传弹窗：拖拽区、预览与分类

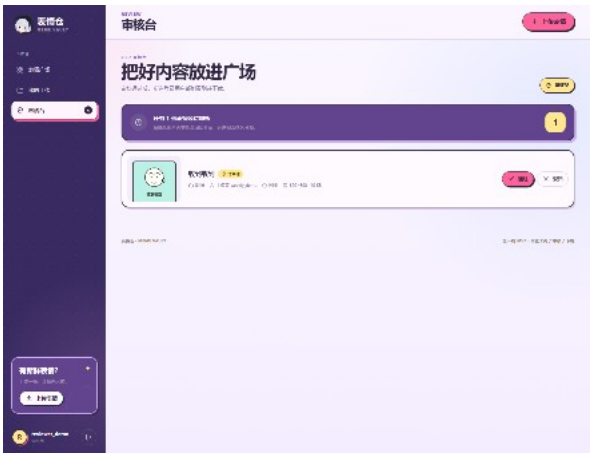


图 1-3 审核台：角色标识与审核卡片

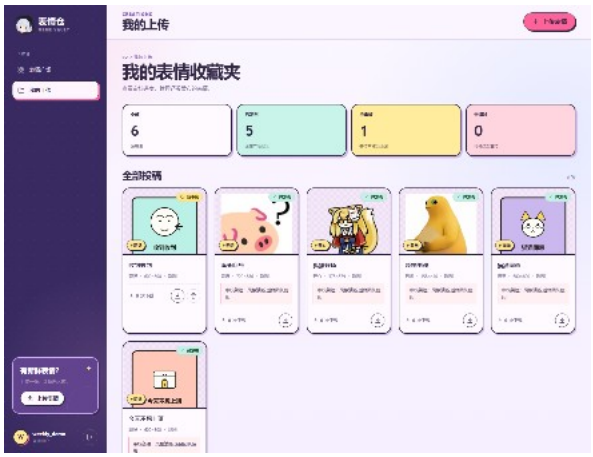


图 1-4 我的上传：统计卡、状态与审核备注

1.5 响应式与交互反馈

布局不是简单缩放：桌面侧栏在 900 px 以下切换为底部导航，搜索框单独占一行，Hero 压缩为更短的标题区，表情网格变为两列；在 340 px 以下才降为单列。上传弹窗在手机端变成底部抽屉，避免小屏弹窗过高。

反馈覆盖主流程的每一步：搜索时显示骨架屏；上传时按钮禁用并显示进度；失败时表单内联错误；审核通过后 Toast + 队列刷新；空列表、空审核队列、图片加载失败都有独立状态。这样用户不会面对“点了没反应”的空白。

1.6 设计验收清单

维度	验收点	结果
视觉层级	首屏 3 秒内能识别“上传”主操作；标题、筛选、卡片层级明确	通过
状态反馈	骨架屏、按钮 loading、空状态、Toast、审核状态均有文字与颜色双重表达	通过
响应式	1440×1100 与 390×844 无横向溢出；移动端底栏与两列卡片正常	通过
可访问性	键盘焦点可见、按钮有 aria-label、prefers-reduced-motion 可降级动效	通过

2 数据库设计

三张表支撑账号、会话与内容；元数据入库，原图落盘，通过外键与状态约束保持一致性。

2.1 设计原则

- **账号与内容分离**：users 只保存身份与角色，emojis 保存业务内容，避免职责混杂。
- **会话可撤销**：数据库只存 token 的 SHA-256 摘要，退出或重置密码时可立即失效。
- **状态显式化**：用 CHECK 约束限定 pending / approved / rejected，避免出现非法审核态。
- **文件不存 BLOB**：数据库只保存 stored_name 与元数据，原图落在 uploads/，降低数据库体积。
- **索引围绕真实查询**：公开列表、个人列表、分类筛选、会话清理各有索引。

2.2 实体关系

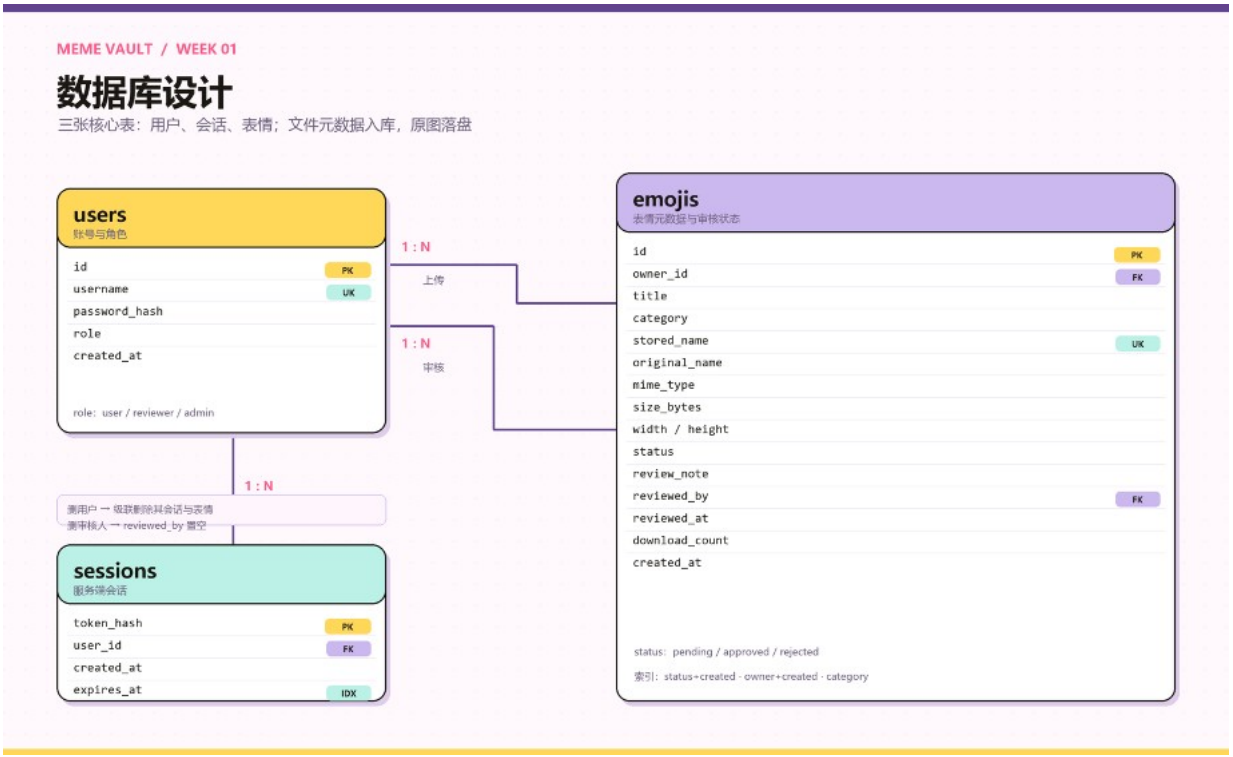


图 2-1 数据库 ER 图：users 1:N sessions，users 1:N emojis（上传与审核）

2.3 表结构

users • 账号与角色

字段	类型	约束	说明
id	INTEGER	PK, AUTOINCREMENT	用户主键
username	TEXT	UNIQUE, COLLATE NOCASE, NOT NULL	2–24 位中英文/数字/下划线
password_hash	TEXT	NOT NULL	scrypt$参数$salt$hash
role	TEXT	CHECK (user/reviewer/admin)	角色
created_at	TEXT	NOT NULL	ISO 8601 UTC 时间

sessions • 服务端会话

字段	类型	约束	说明
token_hash	TEXT	PK	随机 token 的 SHA-256 摘要
user_id	INTEGER	FK → users.id, ON DELETE CASCADE	所属用户
created_at	TEXT	NOT NULL	签发时间
expires_at	TEXT	NOT NULL, INDEX	默认 7 天有效期

emojis • 表情元数据与审核状态

字段	类型	约束	说明
id	INTEGER	PK, AUTOINCREMENT	表情主键
owner_id	INTEGER	FK → users.id, CASCADE	上传者
title	TEXT	NOT NULL	标题，最多 60 字符
category	TEXT	NOT NULL DEFAULT '未分类'	分类，最多 20 字符
stored_name	TEXT	UNIQUE, NOT NULL	UUID + 真实扩展名
original_name	TEXT	NOT NULL	清洗后的原文件名
mime_type	TEXT	NOT NULL	服务端识别的 MIME
size_bytes	INTEGER	NOT NULL	文件字节数
width / height	INTEGER	NOT NULL	像素尺寸
status	TEXT	CHECK (pending/approved/rejected)	审核状态
review_note	TEXT	NULL	审核备注，最多 200 字
reviewed_by	INTEGER	FK → users.id, SET NULL	审核人
reviewed_at	TEXT	NULL	审核时间

字段	类型	约束	说明
download_count	INTEGER	NOT NULL DEFAULT 0	下载次数，原子自增
created_at	TEXT	NOT NULL	上传时间

2.4 索引与查询路径

索引	对应查询	设计意图
idx_sessions_expiry	清理过期会话	定时任务按 expires_at 删除，避免全表扫描
idx_emojis_status_created	公开列表 / 审核队列	先筛状态再按时间排序
idx_emojis_owner_created	我的上传	按上传者过滤并按最新排序
idx_emojis_category	分类筛选 / 分类统计	支撑分类聚合与筛选

2.5 关系、删除策略与一致性

- **users → sessions**：一对多，删用户级联删会话；会话过期由定时任务清理。
- **users → emojis（owner）**：一对多，删用户级联删除其表情记录；文件在业务层同步清理。
- **users → emojis（reviewer）**：一对多，删审核人时 reviewed_by 置空，保留内容与审计线索。
- **上传一致性**：先校验并写文件（`wx` 独占创建），再插入数据库；数据库失败则删除文件。
- **收藏/删除一致性**：公开列表只查 approved；未通过内容仅本人、审核员、管理员可见；owner 可删 pending/rejected。
- **并发与持久化**：开启 WAL、foreign_keys、5 秒 busy_timeout；下载计数用 `download_count + 1` 原子更新。

> **当前边界**
>
> - SQLite 适合单机首周；多实例部署时需要换成 PostgreSQL 或托管数据库。
> - 本地文件目录适合单节点；后续可迁移到对象存储，并补充孤儿文件清理任务。
> - 当前没有审计日志表与软删除；下一阶段建议增加 review_logs 与 deleted_at。

3 公共类设计

不为了“像 OOP”而堆类；把跨页面、跨接口复用的能力沉淀为异常类、模块和纯函数。

3.1 服务端统一异常类：ApiError

后端没有引入 ORM 或框架，唯一的核心业务类是 `ApiError`。它把 HTTP 状态码、稳定错误码、可展示文案与可选细节封装在一起，路由抛出后由顶层统一序列化为 `{ error: { code, message } }`，避免每个接口重复拼错误响应。

```
class ApiError extends Error {
  constructor(status, code, message, details) {
    super(message);
    this.name = 'ApiError';
    this.status = status;
    this.code = code;
    this.details = details;
  }
}
```

属性 / 方法	作用	示例
status	HTTP 状态码	401 / 403 / 409 / 413
code	稳定业务错误码	INVALID_CREDENTIALS / ALREADY_REVIEWED
message	可直接展示给用户的中文文案	登录尝试过多，请 15 分钟后再试
details	可选结构化信息	字段校验细节（当前大多省略）
统一 catch	区分 ApiError 与未知异常	4xx 原样返回；500 记录日志并隐藏细节

3.2 后端公共模块

模块	主要职责	被谁复用
config.js	环境变量、路径、上传上限、Cookie 安全策略	server / app / create-user
database.js	SQLite 连接、建表、索引、初始账号	应用启动与测试
security.js	scrypt 密码、token 哈希、Cookie 解析/序列化、用户名与密码校验	注册、登录、鉴权、CLI
images.js	PNG/JPEG/GIF/WebP 文件头解析、尺寸读取、文本与文件名清洗	上传接口
app.js helpers	JSON 响应、错误响应、请求体限长、同源判断、客户端 IP、分页解析	所有 API 路由
create-user.js	创建账号 / 重置密码 / 修改角色，并撤销旧会话	部署与运维

3.3 前端公共层

前端是一个原生单页应用，通过 `state` + 纯函数 + 事件委托控制界面。没有组件框架，但公共交互被抽成可复用函数，避免各页面重复实现。

公共函数 / 对象	职责	复用场景
state	当前用户、视图、列表、筛选、上传与审核状态	所有页面共享
api()	同源 fetch、JSON/FormData、401 失效、错误标准化	13 处业务请求
el() / icon() / buttonWithIcon()	安全创建 DOM、SVG symbol、按钮工厂	所有动态渲染
createMedia() / makeEmojiCard()	缩略图、分类标签、状态徽章、下载按钮	广场与我的上传
setFormError() / setButtonBusy()	表单错误、按钮禁用与 loading 文案	登录、上传、审核、下载
toast() / clearToasts()	统一轻提示与切换账号清理	所有操作反馈
formatRelative() / formatBytes() / formatFullDate()	时间、体积、绝对时间格式化	卡片、审核、我的上传
dialog 事件封装	打开、关闭、ESC、遮罩点击、上传中阻止关闭	上传与驳回弹窗

3.4 CSS 公共层与测试公共类

CSS 公共层：通过 `:root` 设计令牌统一颜色、圆角、字体与阴影；按钮、表单、卡片、状态胶囊、空状态、Toast、弹窗等以独立类复用。后段主题覆盖只改变量，不重写组件结构，因此换肤成本低。

测试公共类：`test/app.test.js` 中的 `Client` 类维护 Cookie、封装 JSON 请求，使注册、上传、审核、下载等用例可以用同一套会话上下文串联，避免每个断言都重复处理认证。

> **设计评价**
>
> - 优点：公共能力边界清晰，代码量小，零依赖，易于在云端单机部署。
> - 取舍：没有引入 DI、Repository、Service 基类；对于当前规模，函数与模块比类层次更直接。
> - 演进：当批量上传、审核日志、对象存储接入后，再将 emoji 业务拆为 repository/service 层。

4 关键流程与接口设计

把权限写进流程，而不是只写在按钮显隐里。

4.1 认证与会话流程

1. 注册时校验用户名/密码格式，使用随机 salt 做 scrypt，数据库只保存哈希。

2. 登录成功后生成 32 字节随机 token，数据库保存其 SHA-256 摘要，浏览器只拿到原始 token 的 HttpOnly Cookie。

3. 每次请求从 Cookie 取 token，哈希后查 sessions，联表取用户角色；过期会话立即删除。

4. 退出登录删除当前会话；管理员重置密码时删除该用户全部会话。

5. Cookie 使用 HttpOnly、SameSite=Strict，生产环境默认 Secure；写请求额外校验 Origin/Sec-Fetch-Site。

4.2 上传、审核与下载状态

状态	可见范围	可下载	可执行操作
pending	本人、审核员、管理员	本人 / 审核员 / 管理员	审核通过/驳回；本人可删除
approved	所有登录用户	所有登录用户	审核员不可重复审核；管理员可删除
rejected	本人、审核员、管理员	本人 / 审核员 / 管理员	本人可删除并重新上传

上传流程：检查 multipart → 限制请求体 → 读取文件 → 校验真实格式与尺寸 → 生成 UUID 文件名 → 写盘 → 插入数据库 → 返回状态。普通用户进入 pending，审核员/管理员上传直接 approved。

下载流程：查询表情 → 判断 approved / 本人 / 审核角色 → 定位文件 → 累加下载次数（仅 GET）→ 以 attachment 返回原图。文件缺失返回 410，未授权返回 403，未登录返回 401。

4.3 接口清单

分组	方法与路径	说明
健康检查	GET /api/health	服务存活检查
认证	POST /api/auth/register	注册并自动登录

分组	方法与路径	说明
认证	POST /api/auth/login	登录，失败限流
认证	POST /api/auth/logout	删除当前会话
认证	GET /api/auth/me	当前用户
表情	GET /api/emojis	公开列表：q/category/sort/page
表情	POST /api/emojis	单张上传 multipart
表情	GET /api/emojis/mine	我的全部投稿
表情	GET /api/emojis/:id/image	受保护预览
表情	GET /api/emojis/:id/download	下载并计数
表情	PATCH /api/emojis/:id	修改标题/分类
表情	DELETE /api/emojis/:id	本人待审或管理员删除
分类	GET /api/categories	已发布分类及数量
审核	GET /api/review/count	待审数量
审核	GET /api/review/queue	待审队列
审核	POST /api/emojis/:id/review	通过 / 驳回

4.4 权限矩阵与错误格式

操作	未登录	普通用户	审核员	管理员
浏览公开列表 / 下载	拒绝	允许	允许	允许
上传	拒绝	允许（待审）	允许（直接发布）	允许（直接发布）
查看待审内容	拒绝	仅自己的	全部	全部
审核	拒绝	拒绝	允许	允许
修改任意内容	拒绝	仅自己的待审	拒绝	允许
删除任意内容	拒绝	仅自己的待审/驳回	拒绝	允许

错误响应统一为 `{ "error": { "code": "...", "message": "..." } }`。常见状态码：400 参数/格式错误、401 未登录、403 越权/跨站、404 不存在、409 冲突、413 文件过大、415 非图片、429 登录过频。

5 分工与 AI 协作

本阶段由项目发起人与 AI 编程代理协作完成；关键取舍与验收由人负责，重复实现与验证由 AI 加速。

口径说明

- 为避免虚构成员，下表按真实参与主体记录，不写未提供的姓名。
- 若提交要求“每人一行”，可使用 5.4 的多人团队模板，将姓名替换为实际成员。
- AI 工具不计为人员，单独列在 5.3，说明其在需求、开发、测试、文档中的具体作用。

5.1 实际参与主体

参与主体	身份 / 角色	本阶段分工	产出与责任
项目发起人（姓名待填）	产品负责人 / 最终验收人	提出表情包共享需求；确定首周最小闭环与优先级；选择 Node + SQLite + 原生前端路线；确认视觉方向；审核安全与部署策略。	需求决策、范围裁剪、验收签字；对最终上线与数据安全负责。
AI 编程代理	开发 / 测试 / 文档协作者（非人类）	把自然语言需求拆成模块与接口；生成数据库、后端、前端与测试初稿；修复沙箱与浏览器问题；执行无头 Edge 闭环；整理本简报。	代码、测试、截图与文档初稿；所有输出仍需人工审查与验收。
审核员 / 管理员	业务角色（可由发起人兼任）	定义“可发布”的标准；实际走审核通过/驳回；验证角色隔离、审核备注和下载权限。	内容质量把关、权限验收；不代表额外开发人力。
自动化工具链	非人类工具	Node 测试运行器、无头 Edge + CDP、Pillow、python-docx 分别承担断言、浏览器操作、图片与文档生成。	可重复的测试与截图证据，不参与产品决策。

5.2 按阶段拆解的分工

阶段	主要工作	人负责	AI / 工具参与
需求与架构	明确角色、范围、数据模型、技术路线	确定“先跑通闭环”的优先级与安全底线	输出需求结构、架构方案、风险清单
视觉与交互	确定紫粉拼贴方向、页面层级、移动端策略	选择方向、否决不合适的方案	编写 CSS、生成布局、迭代截图

阶段	主要工作	人负责	AI / 工具参与
后端与数据	认证、上传、审核、下载、SQLite 表结构	审查权限边界与数据一致性	生成路由、SQL、哈希与文件校验代码
前端联调	登录、广场、上传、审核、我的上传	确认交互与信息层级	实现 DOM、状态、事件、API 对接
测试与验收	9 项集成测试、真实浏览器闭环、移动端检查	判断是否达到可演示/可部署标准	编写测试、启动 Edge、截图、定位问题
文档与部署	README、环境变量、Nginx/HTTPS、持久化清单	确认云端部署策略	整理文档、生成简报、绘制图表

5.3 AI 工具在本阶段的作用

工具	本阶段用途	具体贡献	人工把关点
DeepSeek Harness（deepseek-flash）	主编程代理	需求结构化、架构与数据库设计、后端/前端实现、调试、测试、文档草拟；把“做表情包网站”拆成可验收的 16 个接口。	范围取舍、技术路线确认、安全审查、最终验收。
Node.js Test Runner	后端自动化验证	9 项断言覆盖注册、上传、待审、审核、公开列表、下载、跨站拦截、退出。	确认测试用例是否覆盖真实风险。
Microsoft Edge + CDP	真实浏览器验收	自动完成注册→上传→审核→展示→搜索→下载；检查桌面/移动布局与横向溢出；生成页面截图。	判断视觉与交互是否符合产品目标。
Pillow	测试素材与图表	生成演示表情、架构图、ER 图；压缩与检查截图。	确认图表信息准确、无错别字。
python-docx	文档排版	生成 Word 简报、封面、表格、页眉页脚、页码。	确认版式、文字口径与提交格式。

5.4 若按多人团队填报

如果课程或团队要求按成员姓名提交，可沿用下表。姓名列留空，按实际小组填写即可；分工与当前代码/文档的产出是一一对应的。

成员姓名	建议角色	可认领的分工	对应产出
(待填)	产品 / 项目经理	需求访谈、范围与优先级、验收标准、进度协调	需求清单、验收记录、简报摘要
(待填)	前端 / 视觉	页面结构、CSS 主题、响应式、交互与可访问性	index.html、styles.css、app.js
(待填)	后端 / 数据库	认证、上传、审核、下载、SQLite 与索引	src/app.js、database.js、security.js
(待填)	测试 / 运维	测试用例、真实浏览器验收、部署与备份	app.test.js、README、部署清单
(待填)	内容审核 / 业务	审核标准、分类规范、驳回理由、权限验收	审核规则、审核台验收记录

5.5 协作复盘

- **人负责“为什么”和“做到哪里”**：首周砍掉批量上传与全局管理 UI，是人工做出的范围决策，避免 AI 把项目做成大而全。
- **AI 负责“怎么快速实现”**：把重复的 CRUD、CSS 细节、测试与截图自动化，显著缩短从需求到可演示版本的时间。
- **截图验收不可省略**：CSS/交互问题很难只靠单元测试发现；无头浏览器截图暴露过按钮拥挤、移动端首屏过长等问题。
- **安全边界必须人工复核**：密码哈希、会话、同源校验、文件类型与权限矩阵均需按真实威胁模型检查。
- **AI 不是成员**：它没有产品判断与责任主体；报告中将 AI 标注为工具/协作者，避免把工具贡献写成虚构人员。

6 验收、风险与下一步

用可重复的测试和真实浏览器闭环证明“能用”，而不是只展示页面。

6.1 自动化测试记录

编号	验证点	断言 / 结果
T1	健康检查与未登录拦截	200 健康；/api/auth/me 返回 401
T2	注册与 Cookie 会话	201；角色 user；Set-Cookie 写入
T3	单张 PNG 上传	201；status=pending；宽高识别为 1×1
T4	公开列表与我的上传	公开 total=0；我的 total=1
T5	审核员登录与通过	200；角色 reviewer；状态 approved
T6	公开列表与下载	total=1；下载返回附件；字节数一致；计数 +1
T7	跨站写请求	Origin 为外部域 → 403
T8	退出登录	会话删除；再次访问列表 → 401
T9	测试隔离	临时目录 + 独立 SQLite，退出后清理

执行命令 `npm test`，结果 **9 项全部通过**。测试使用临时数据库和上传目录，不污染开发数据。

6.2 真实浏览器闭环

Edge 无头浏览器验收

- 注册普通用户 → 上传 PNG → 我的上传显示待审核。
- 审核员登录 → 审核台看到待审内容 → 通过。
- 普通用户重新登录 → 广场出现内容 → 搜索命中 → 下载原始文件。
- 浏览器实际下载 57,906 字节，与上传文件一致；桌面 1440×1100、移动 390×844 均无横向溢出。
- 唯一的控制台 401 来自首次 /api/auth/me 的未登录探测，属于预期行为。

6.3 风险与应对

风险	影响	当前应对	下一阶段
上传恶意文件	存储或浏览器安全	只收真实位图，校验文件头/尺寸/大小，CSP 与 nosniff	接入病毒扫描、对象存储隔离

风险	影响	当前应对	下一阶段
SQLite 单写者	并发增长时写入等待	WAL、busy_timeout、短事务	迁移 PostgreSQL
本地磁盘单点	多实例/灾备不足	持久化卷 + 备份说明	对象存储 + CDN
管理员 UI 不完整	全局治理效率低	后端 PATCH/DELETE 已具备，CLI 可建账号	管理后台、审计日志
登录暴力尝试	账号风险	IP+用户名 8 次/15 分钟限流	验证码、设备管理、告警
备份不一致	数据库与文件对不上	data/ 与 uploads/ 一起备份的部署要求	快照与恢复演练

6.4 下一阶段路线

优先级	事项	价值	预估工作量
P0	批量上传与批量审核	完成“添加表情包（批量）”的原始需求	中
P0	管理员全局内容管理页	修改/删除所有内容，补齐后台治理	中
P1	分类管理 + 标签	让分类从自由文本升级为可维护体系	中
P1	ZIP 批量下载	提升收藏效率	低
P1	用户与角色管理、审计日志	云端运营与追责	中
P2	对象存储、缩略图、CDN	多实例与性能	高
P2	密码找回与登录设备管理	账号安全体验	中

6.5 结论

第一周的核心成果不是 16 个接口或 3 张表，而是**把“谁可以上传、内容如何审核、通过后如何被看见和下载”变成了可运行、可验证、可部署的系统。**前端有完整自定义 CSS 与响应式体验，数据库有明确关系和索引，公共层保持克制且可复用，测试覆盖真实业务风险。

下一阶段应优先补齐批量能力与管理员治理界面，同时把 SQLite/本地文件替换为更适合云端的数据库与对象存储。当前版本已经足够作为演示、验收和继续迭代的基础。

附录

接手、运行与部署所需的最少信息。

A 本地运行

```
# 要求 Node.js >= 22.5
npm start
# 打开 http://127.0.0.1:3000
# 首次启动创建开发演示账号：
#   admin / Admin@123456
#   reviewer / Review@123456

npm test        # 9 项后端集成测试
node scripts/create-user.js   # 需配合 NEW_USER_PASSWORD 环境变量
```

项目没有运行时 npm 依赖，不需要 `npm install`。`.env` 可选，`npm start` 会自动读取存在的 `.env`；生产环境需设置 `NODE_ENV=production`、`COOKIE_SECURE=1`、`SEED_DEMO_ACCOUNTS=0` 或强密码。

B 目录索引

路径	作用
public/index.html	登录页 + 单页应用结构 + 内联 SVG 图标
public/styles.css	主题令牌、布局、组件、动效、响应式
public/app.js	状态、API 封装、页面渲染、上传/审核/下载交互
src/app.js	HTTP 路由、权限、上传与下载、审核、错误处理
src/database.js	SQLite 表结构、索引、初始账号
src/security.js	scrypt、token、Cookie、输入校验
src/images.js	图片格式与尺寸识别、文本清洗
test/app.test.js	9 项后端集成测试与 Client 测试助手
scripts/create-user.js	创建账号 / 重置密码 / 更新角色
docs/screenshots	早期界面截图（归档）
report/screenshots	本简报使用的当前界面截图
report/diagrams	架构图与 ER 图

C 部署检查清单

检查项	要求
账号	关闭示例账号，使用 create-user 创建强密码管理员/审核员
HTTPS	Nginx/Caddy 终止 TLS，COOKIE_SECURE=1
持久化	data/ 与 uploads/ 挂载持久卷并纳入备份
代理	TRUST_PROXY=1，client_max_body_size ≥ 6 MB
权限	uploads/ 不作为静态目录暴露
验证	上线后实际走一遍注册、上传、审核、下载

文档结束

- 本简报基于 E:\wwweb 当前代码、测试结果与 2026-10-30 截图整理。
- 团队成员姓名与实际分工如与本报告口径不同，可在 5.1 / 5.4 表中替换后提交。